다섯 가지 의견 모두 방향은 맞습니다. 다만 두 가지는 먼저 정리해야 합니다. 테스트와 운영의 차이를 DB 데이터를 고쳐서 만든 점, 그리고 10,000건이 **개발계정 기준 한도**라는 점입니다. 운영 설계는 운수사 시간표를 받아서 정하면 되고, 지금은 테스트 구조부터 바로잡기를 권합니다.

1. 의견별 검토

의견	검토
1. PC 테스트 + 실제 차량번호	좋은 방식입니다. 단, 실제 차량이라서 **운행하지 않는 시간에는 위치가 없습니다.** 밤이나 운행 사이의 404는 버그가 아니라 정상입니다. Codex가 이 시간대의 결과를 보고 버그라고 판단하면 말이 오락가락할 수 있어서, AGENTS.md에 적었습니다
2. 운영 방식(차량번호로 노선 찾기 → 다음 정류장 도착정보)	맞습니다. TAGO 위치 API는 노선 단위로만 조회되고 차량번호로 바로 찾는 기능이 없습니다. 그래서 **"이 차가 어느 노선에 있나"를 찾는 단계가 가장 많은 호출을 씁니다**
3. 105개 노선 순회 → 3대 고정	계산이 맞습니다. 10초 간격이면 105 × 10초 = 17.5분입니다. 실제로는 아래 4번 이유로 약 35분입니다. 3대로 줄인 판단은 맞지만, **DB 데이터를 고쳐 고정한 것**이 테스트와 운영이 꼬인 원인일 가능성이 큽니다. 아래 2장에서 다룹니다
4. 10초 간격에 10초 대기가 또 붙음	맞습니다. 스케줄러가 작업이 끝난 뒤 10초를 쉬고, 호출 직전에 공용 대기 장치가 다시 "두 API를 합쳐 10초에 1번"을 지킵니다. 위치·도착 수집기가 둘 다 켜져 있으면 **API마다 약 20초에 1번**만 호출됩니다. 그래서 3대 테스트에서도 노선 하나가 약 60초마다 갱신됩니다. 제안한 새 구조는 스케줄러가 1초마다 확인만 하고, 실제 간격은 한도 관리 장치(guard) 하나만 정합니다. 이 원칙을 AGENTS.md에 명시했습니다
5. 운행 시간표로 호출 줄이기	가장 효과가 큰 방법입니다. 다만 지연·결행·임시 배차가 있으니 **시간표로는 "언제 찾기 시작할지"만 정하고, 실제로 운행하는지는 TAGO 응답으로 확인**해야 합니다

추가로 확인된 점: 공공데이터포털의 두 API 모두 "개발계정 10,000건 / 운영계정은 활용사례를 등록한 뒤 신청하면 트래픽 증가 가능"으로 안내되어 있습니다(버스위치정보, 버스도착정보). 얼마나 늘려 주는지는 안내되어 있지 않습니다. 그래서 설계는 10,000건에 맞추고, 운영을 시작할 때 증설을 신청하는 것을 권장합니다.

2. 지금(테스트 단계) 제안

1. **테스트와 운영의 차이는 "수집 대상 목록"과 "설정값"만 두기.** DB의 노선이나 정류소 데이터를 3대에 맞게 고치는 대신, 수집할 차량 목록(예: 단말 배정 테이블이나 설정)만 3대로 둡니다. 그래야 운영으로 넘어갈 때 코드가 바뀌지 않습니다.
    - 실제 차량은 하루 중 노선을 바꿀 수 있습니다. 노선 데이터를 고정해 두었다면, 차량이 다른 노선으로 가는 순간 "위치 없음"이 됩니다.
2. **고정한 방식부터 Codex에게 확인시키기.** 수정하지 말고 보고만 하라고 시키세요(AGENTS.md "현재 단계" 섹션).

    > "AGENTS.md의 '현재 단계' 섹션의 '먼저 보고'를 수행해줘. 수정은 하지 마."

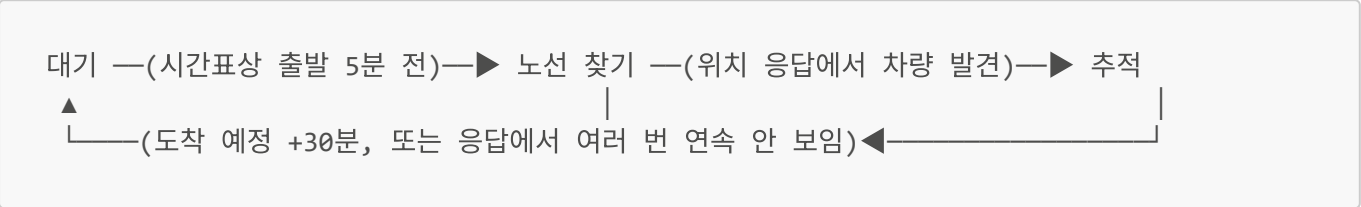
    테스트용 데이터가 Flyway 마이그레이션 파일 안에 있으면 운영 DB에도 그대로 들어갑니다. 이 부분을 꼭 확인해야 합니다.

3. **테스트 설정:** 24시간, API별 10초 간격, 하루 8,640건입니다. 대기 장치 하나만 간격을 정하므로 실제로도 10초가 됩니다.

4. **운영 서버를 띄우면 PC의 수집기는 끄기**(`tago.enabled=false`). 같은 서비스키를 쓰면 한도를 나눠 쓰게 됩니다.

3. 운영 단계 제안 (운수사 자료를 받은 뒤)

차량별 상태 흐름



- **노선 찾기:** 후보 노선을 좁혀서 조회합니다. 우선순위는 배차표에 있는 노선 → 시간표상 지금 운행 중인 노선 중 이 차량이 예전에 탔던 노선 → 지금 운행 중인 전체 노선 순입니다. 위치 API는 한 번 호출로 그 노선의 모든 차량을 돌려주므로, 찾고 있는 차량이 여러 대여도 한 번에 함께 확인됩니다.
- **추적:** 단말이 있는 차량이 운행 중인 노선만 조회합니다.
- **도착정보:** 다음 정류장이 바뀔 때 1번만 조회하고, **도착 예정 시각을 저장해 화면에서 남은 분을 줄여 갑니다**(P1-4에 이미 반영). 정류장마다 반복 조회할 필요가 없습니다.

예산 계산 (가정: 차량 1대가 하루 8회, 1회 1시간, 정류장 30개)

항목	차량 1대당 하루 호출	한도 9,500건으로 가능한 대수
위치 추적 (30초 간격)	8시간 × 120 = 960건	약 **9대** (서로 다른 노선일 때)
위치 추적 (60초 간격)	480건	약 19대
도착정보 (정류장마다 1회)	8 × 30 = 240건	약 39대
노선 찾기 (시간표 사용 시)	운행 시작 전후 몇 건	거의 영향 없음

→ **개발계정 한도로는 단말 약 10~20대 규모가 한계이고, 위치 API가 먼저 바닥납니다.** 그보다 많으면 다음 중 하나가 필요합니다.

- **트래픽 증설 신청**
- **단말 GPS 활용:** 차량 단말이 직접 자기 위치를 서버로 보내면, TAGO 위치 API는 노선을 확인할 때만 쓰게 됩니다. 이 경우 위치 호출이 거의 사라지고, 도착정보 기준으로 약 39대까지 늘어납니다. 위치 갱신도 TAGO보다 빨라집니다.

운영 패턴 파악: "관측 모드" (선택)

운수사 자료와 별개로, 테스트 기간 중 며칠(평일·주말 각각)은 3대 테스트를 쉬고 105개 노선을 10초 간격으로 돌며 **차량–노선–시각을 기록**하는 방법이 있습니다.

- 한 바퀴가 17.5분이라 1시간짜리 운행은 거의 빠짐없이 잡힙니다.
- 실제 배차 패턴을 데이터로 확보할 수 있고, 운수사 시간표와 비교해 검증할 수도 있습니다.
- 그날 위치 API 한도는 대부분 이 기록에 씁니다.

4. 운수사 협의 때 받을 자료 (효과가 큰 순서)

1. **차량별 배차표:** 어느 차량이 몇 시에 어느 노선을 도는지. 이게 있으면 노선을 찾는 단계가 사라집니다
2. **노선별 시간표:** 평일·토·일·공휴일 구분, 출발 시각, 1회 운행 시간
3. 차량이 노선을 바꾸는 규칙(오전·오후 교대 등)과 예비차 운영 방식
4. 막차가 실제로 끝나는 시각(20:30이 마지막 출발 시각인지, 운행 종료 시각인지)

5. 정해 주실 것

1. 차량 단말에 **GPS가 있는지**, 지금 앱이 GPS를 쓰는지
2. 운영할 때 **단말 대수**(예상)
3. **관측 모드**를 진행할지

답을 주시면 운영 단계 설계를 AGENTS.md에 P4로 추가하겠습니다. 지금은 Codex가 운영 로직을 미리 만들지 않도록 "구현하지 않음"으로 막아 두었습니다.

AGENTS.md 변경 (rev 9)

- **"현재 단계: 테스트" 섹션 추가(44–49줄):** 단말 3대, 24시간·10초 수집, 운행하지 않는 시간의 404는 정상, 운영 로직은 아직 구현하지 않음, 고정 방식 먼저 보고
- **설정값 정리(65줄):** 테스트는 `min-gap-ms=10000`, `window=00:00-23:59`로 정했습니다. 예전 값 `05:00-23:30`은 테스트 결정(24시간)과도, 실제 운행 시간과도 맞지 않아 혼란의 원인이 될 수 있었습니다
- **두 번 기다리지 않기(68줄):** 스케줄러 간격과 한도 관리 장치의 간격을 겹치지 않게 했습니다
- **배경(37줄):** 10,000건이 개발계정 기준임을 적었습니다

프로젝트 폴더의 `AGENTS.md`도 이 내용으로 바꿔 넣어 주세요.

AGENTS.md